

Block 2

Your First webMethods Container

Registry · MSR image · Health · Logs · application.properties

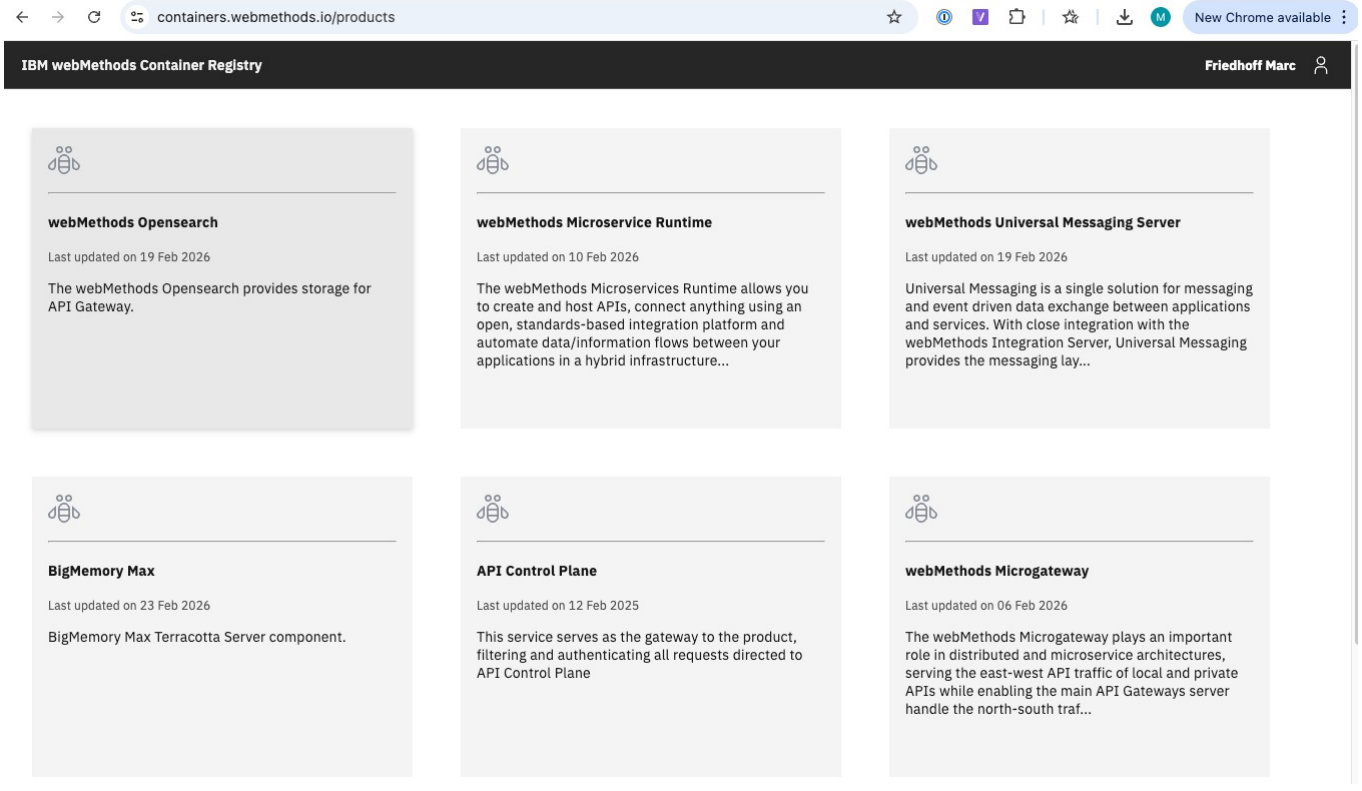
IBM webMethods Images

Container Registry · Access · Run

IBM webMethods Container Registry

IBM Expert Labs

→ https://containers.webmethods.io

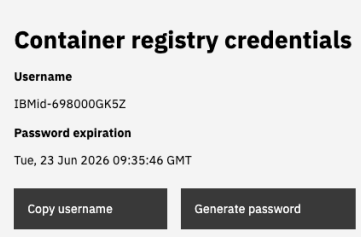


Prerequisites:

PAO – Self Nomination

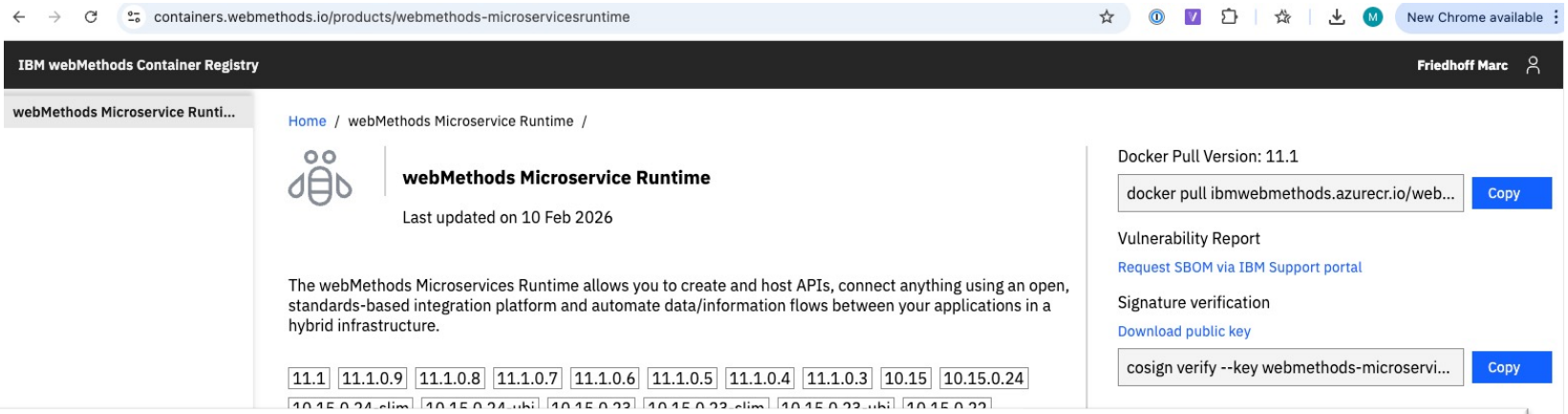
- https://www.ibm.com/software/passportadvantage/

Containers > Settings >



IBM webMethods Container Images

Find Setup Instructions for Container Image



How to use these images

Starting an empty Microservices Runtime

You can startup an empty IBM webMethods Microservices Rutime (MSR) by simply running the container:

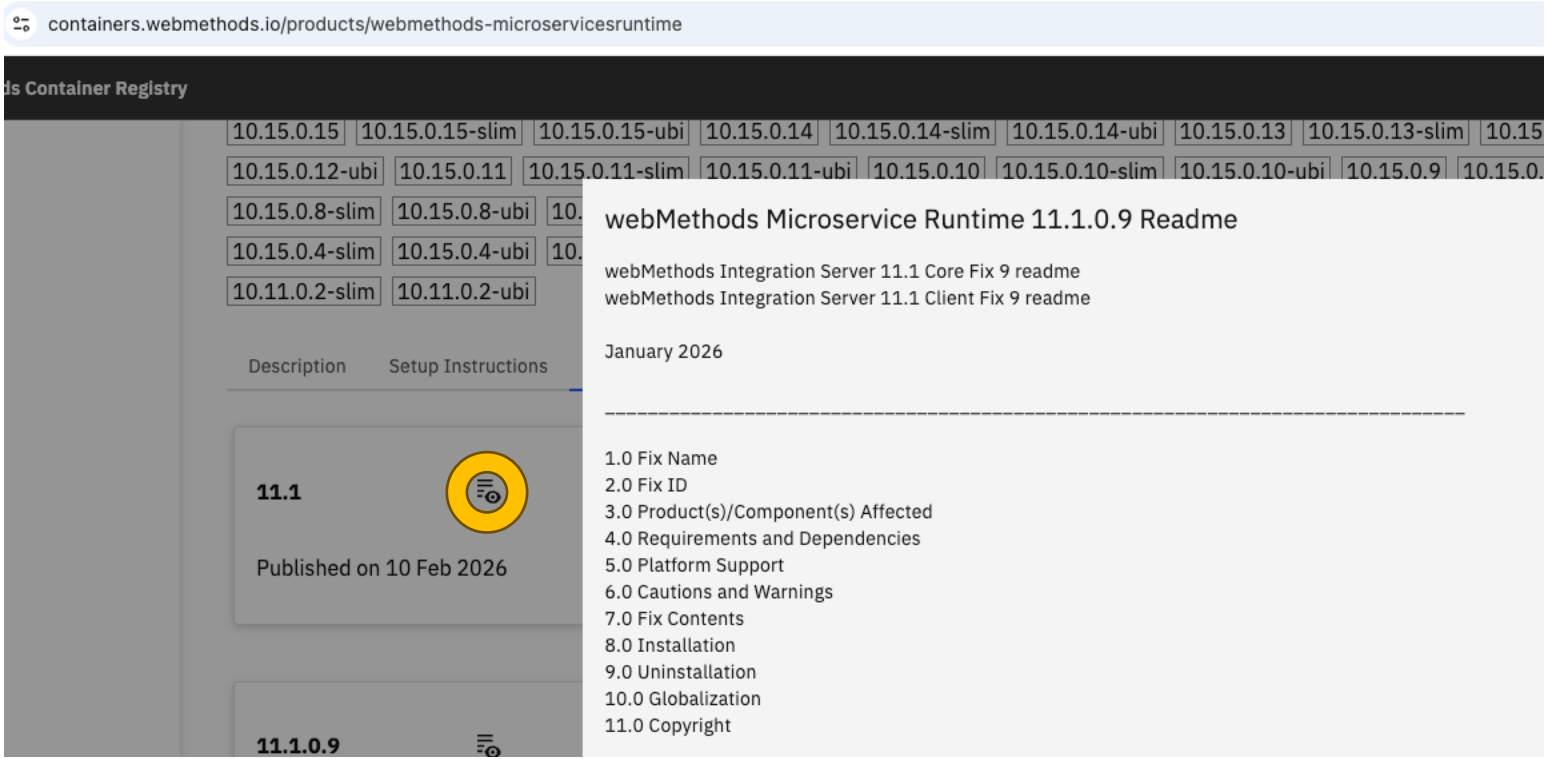
```
docker run -dp 5555:5555 ibmwebmethods.azurecr.io/webmethods-microservicesruntime:10.15
```

This will start up an empty MSR with the default port exposed. You can then use the webMethods client tooling to develop or deploy an application on the MSR.

You can also connect with your browser using http://:5555 (User: Administrator Password:manage) to view the admin console.

IBM webMethods Container Images

Find Readmes



🔧 Exercises 2.1–2.2: Pull and Run the MSR

- Log in: podman login ibmwebmethods.azurecr.io (IBMid + token from containers.webmethods.io)
- Pull: podman pull ibmwebmethods.azurecr.io/webmethods-microservicesruntime:12.1.0.3
- Inspect first: podman image inspect … (user, entrypoint, ports, env); podman image history

→ Hint: the course pins the fix tag 12.1.0.3; the floating tag 12.1 moves to every new IBM fix

Classification - Internal

Official IBM webMethods Images (12.1)

Component	Image (ibmwebmethods.azurecr.io/...)	Used in
Microservices Runtime	webmethods-microservicesruntime:12.1.0.3	Blocks 2–6 (base of our custom image)
Integration Server	webmethods-integrationserver:12.1.0.3	Reference: full IS, not used in the capstones
Universal Messaging	universalmessaging-server:12.1.0.0	Capstones 4–6
API Gateway	apigateway-minimal:11.1.0.14 (no 12.1 image yet)	Capstones 5–6 (needs an external OpenSearch)
Microgateway / Dev Portal	microgateway:11.1.0.14 / devportal:11.1.0.14	Outlook
Active Transfer / Trading Networks	webmethods-activetransfer:12.1.0.1 (+ -dcc); TN: none	Day 3 discussion

Login: podman login ibmwebmethods.azurecr.io (IBMid + token from containers.webmethods.io). Pin tags, mirror into the Swisscom registry.

Classification - Internal

Integration Server vs. Microservices Runtime

Integration Server (classic)

- Multiple instances under instances/<name>
- Typically hundreds of packages per server
- Configured interactively or via Ansible
- Long-lived servers, patched in place
- Image exists (webmethods-integrationserver) but is heavier

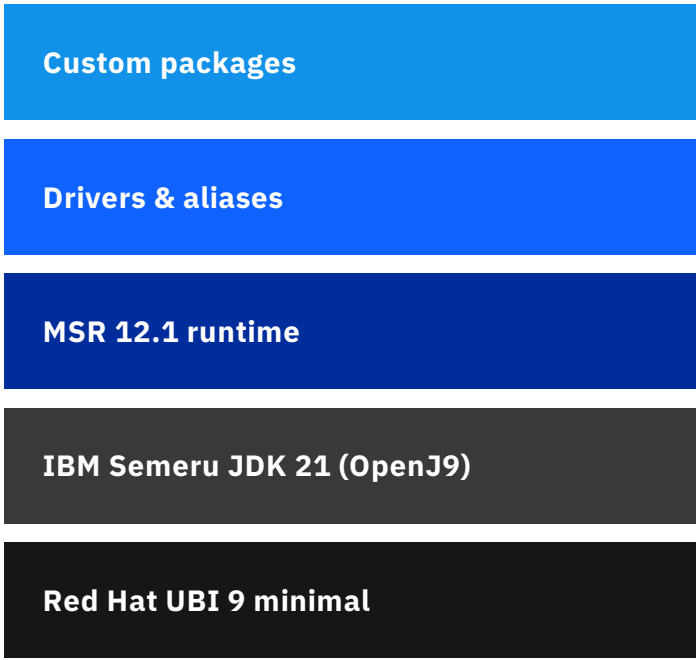
Microservices Runtime (this workshop)

- Single runtime directly in /opt/webmethods/IntegrationServer
- Built for containers: 1 to n packages per image
- Configured as code: application.properties + env vars
- Immutable images, replaced instead of patched
- Same packages, same services, same Designer

The ~400 IS packages run on MSR unchanged; what changes is how they are packaged, configured and operated.

Classification - Internal

Anatomy of the MSR Image and Our Custom Image



packages/SwisscomLab (later: the Swisscom packages)

lib/jars/custom (e.g. PostgreSQL JDBC), config/jdbc/driver

/opt/webmethods/IntegrationServer, wpm package manager, UID 1724

/opt/webmethods/jvm/jvm: full JDK incl. javac

Base OS of the IBM image (microdnf, no package manager bloat)

Bottom three layers come from IBM; pull them, never modify them.

Top two layers are ours, built FROM the IBM image in a Containerfile.

application.properties and secrets are NOT layers: they are mounted at runtime.

Classification - Internal

Inside the MSR 12.1 Image

Item	Value
Install directory	/opt/webmethods/IntegrationServer (no instances/default in MSR)
User	sagadmin, UID/GID 1724; the image never needs root at runtime
Ports	5555 HTTP, 5543 HTTPS (enabled by default), 9999 diagnostic (auth required)
Entrypoint	bin/startContainer.sh (traps SIGTERM and calls bin/shutdown.sh)
Health / metrics	/health, /health/liveness, /health/readiness, /metrics (anonymous)
Configuration	/opt/webmethods/IntegrationServer/application.properties (or SAG_IS_CONFIG_PROPERTIES)
Logs	logs/ directory; server log also on stdout (WM_IS_LOG_DESTINATION=both)
Tools in image	curl, JDK 21 (javac), wpm, bin/health-check.sh

Classification - Internal

🔧 Hands-On: Exercises 2.3–2.4

2.3	Inside MSR: id (UID 1724), $SAG_HOME, no instances/, $JAVA_HOME/bin/java -version, PID 1 = startContainer.sh
2.4	/health/liveness, /health/readiness, /health, /metrics; Admin UI: packages, ports, extended settings
📁 Files	ibmwebmethods.azurecr.io/webmethods-microservicesruntime:12.1.0.3 (container msr-first)

Participant Workbook, Block 2 · ~25 min · 🏁 health answers without a password; ping as Administrator returns 200

Classification - Internal

Environment Variables the MSR Image Understands

Variable	Effect
SAG_IS_ADMIN_PASSWORD	Sets the Administrator password at start (default 'manage' is then rejected)
JAVA_MIN_MEM / JAVA_MAX_MEM	Heap size (-ms/-mx); defaults 256M / 1024M
JAVA_CUSTOM_OPTS	Additional JVM options, e.g. -XX:MaxRAMPercentage (OpenJ9 VM: HotSpot-only flags do not apply)
TZ	Time zone of the logs, e.g. Europe/Zurich
SAG_IS_CONFIG_PROPERTIES	Alternative path of application.properties
SAG_IS_LICENSE_FILE	Path to a mounted licence file (without it: unlicensed evaluation mode)
WM_IS_LOG_DESTINATION / LOG_FORMAT	Server log to stdout, file or both (default both); LOG_FORMAT=json for structured logs
SAG_IS_SHUTDOWN_WAIT_TIME	Seconds shutdown.sh waits for running services (default 300)

Classification - Internal

Example: podman run for a Dev MSR

terminal

```
podman run -d --name msr-capstone2 \
  -p 5555:5555 -p 5543:5543 \
  -e SAG_IS_ADMIN_PASSWORD="Init1al!Change" \
  -e TZ=Europe/Zurich -e JAVA_MIN_MEM=512M -e JAVA_MAX_MEM=1024M \
  -v $PWD/msr-capstone2-logs:/opt/webmethods/IntegrationServer/logs \
  --health-cmd='curl -fs http://localhost:5555/health/readiness || exit 1' \
  --health-interval=10s --health-retries=12 --health-start-period=60s \
  ibmwebmethods.azurecr.io/webmethods-microservicesruntime:12.1.0.3
```

The health flags turn "is it up yet?" into a queryable container state (podman ps shows healthy).

Classification - Internal

Health and Metrics Endpoints

/health/liveness	Is the JVM alive? Stays 200 even when UM or the DB is down. Use for restart decisions
/health/readiness	Can it serve? 503 when JDBC, JMS or UM indicators are DOWN. Use for healthchecks and load balancers
/health	Full JSON report: Adapters, Diskspace, JDBC, JMS, Memory, ServiceThread, Sessions, UMAliases
/metrics	Prometheus format: sag_is_used_memory_bytes, sag_is_server_threads, … (Nagios/OP5 on Day 3)

All four are anonymous by default (SAG_IS_HEALTH_ENDPOINT_ACL / SAG_IS_METRICS_ENDPOINT_ACL change that).

Classification - Internal

Port Mapping & Network Configuration

- -p 15555:5555 maps host port 15555 to the container's 5555; host and container ports needn't match
- Two containers can't both publish host port 5555; pick distinct host ports or let Compose handle it (Day 2)
- Admin UI reachable = host port published + container healthy + nothing else on that port
- 5543 (HTTPS) is enabled by default with a self-signed certificate; curl needs -k
- Inside a Compose network services talk on the CONTAINER port (msr-1:5555); host ports don't matter there

Gotcha: 'Up' in podman ps is not 'ready'. MSR needs ~25–75 s; wait for /health/readiness or the healthy state.

Classification - Internal

Log Analysis Inside a Container

- podman logs msr-capstone2 shows the server log; MSR writes it to stdout AND logs/server.log (lines prefixed ISSERVER||)
- Only on disk under /opt/webmethods/IntegrationServer/logs: WMSECURITY_*.log, FailedAudit_*.log, stats.csv
- logs/configurationvariables.log lists every change application.properties made at startup
- Read them with podman exec msr-capstone2 tail -f .../logs/server.log, or mount logs/ and use your editor
- LOG_FORMAT=json gives structured lines for Graylog; Day 3 ships logs centrally instead of tailing

> **Best practice:** If something is missing from podman logs, it is almost always in a file under logs/. Look there before assuming the container is broken.

Classification - Internal



Configuration with MSR

Application.properties

application.properties: Configuration as Code

application.properties

```
# mounted at /opt/webmethods/IntegrationServer/application.properties
user.Administrator.password=$env{ADMIN_PASSWORD}
globalvariable.LAB_ENVIRONMENT.value=$env{LAB_ENV}
settings.watt.server.audit.logFilesToKeep=7

jdbc.LabPostgres.dbURL=$env{DB_URL}                    # Day 2
jdbc.LabPostgres.driverAlias=PostgreSQL_JDBC_Driver
messaging.IS_UM_CONNECTION.um_rname=$env{UM_URL}      # Day 2
jms.LAB_UM_JMS.jndi_automaticallyCreateUMAdminObjects=true
```

Applied on every start and logged as ISS.0028.0007I in logs/configurationvariables.log. One file for all environments; only the environment variables differ.

Classification - Internal

MSR Configuration — application.properties

IBM Expert Labs

Why application.properties?

- Users, global variables, JDBC pools, messaging and extended settings in one file. $env{VAR} keeps one file for all environments. Mounted at runtime: no rebuild.

Users & globals

user.<name>.password
globalvariable.<n>.value

Extended settings

settings.watt.server.*

JDBC & JNDI

jdbc.<pool>.dbURL
jndi.<alias>.providerURL

Messaging & JMS

messaging.<alias>.um_rname
jms.<alias>.*

Volume Mount (Compose)

```
volumes:
 - ./application.properties:\
    /opt/webmethods/IntegrationServer/
    application.properties:ro
```

Benefits

✓ Same image, different config
✓ No rebuild for config changes
✓ Version-controlled settings
✓ Environment-specific tuning
✓ Read-only mount (:ro) for security

Classification - Internal

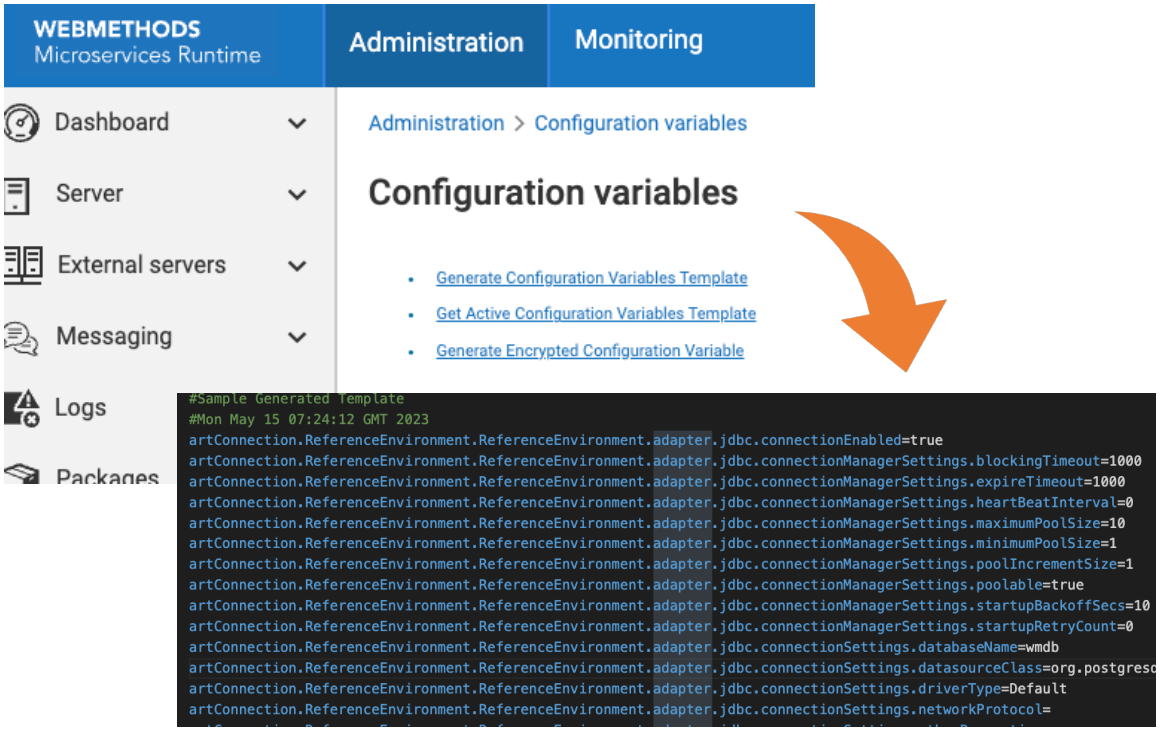
application.properties: Asset Types You'll Use

Prefix	Configures	Example
user.	Users and passwords	user.Administrator.password=$env{ADMIN_PASSWORD}
globalvariable.	Global variables (optionally secure)	globalvariable.LAB_DB_PASSWORD.isSecure=true
settings.	Extended settings (watt.*) in server.cnf	settings.watt.net.timeout=120
jdbc.	JDBC connection pools	jdbc.LabPostgres.maxConns=10
messaging.	UM native messaging aliases	messaging.IS_UM_CONNECTION.enabled=true
jndi. / jms.	JNDI providers and JMS connection aliases	jndi.LAB_UM_JNDI.providerURL=nsp://um:9000

$env{NAME} reads an environment variable. Inject environment variables instead of saving them in config files!

Classification - Internal

MSR Configuration — Templates



Create configuration variable templates from the MSR:

MSR Admin > Microservices > Generate Configuration Variable Template

- Provide as file mount in Container

Classification - Internal

IBM webMethods Service Designer

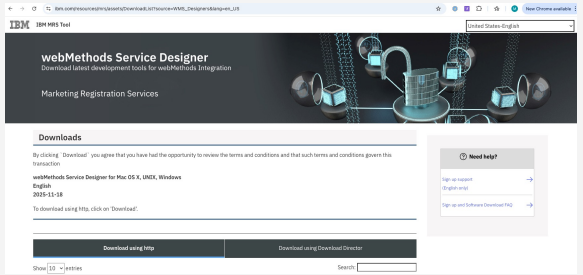
Installation · Connect to Containers · Run

IBM webMethods Service Designer

Download

TechXChange Community Article

Download IBM webMethods Service Designer

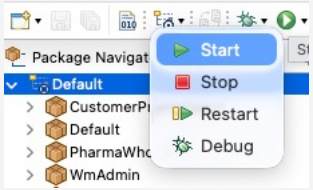


Install

Extract ZIP, run setup

Run

Service Designer comes with local IS and Local Development Plugin, Start and Stop Embedded in Eclipse



🏆 Capstone 2: MSR Configured as Code

Objectives

80 min

- Run MSR with admin password, TZ, heap and a readiness healthcheck
- Log analysis: podman logs vs. the log files; bind-mount the logs folder
- Lifecycle: stop with default 10 s (exit 137) vs. -t 90 (exit 0, ~13 s)
- Complete block2/capstone/application.properties: $env{} password, global variable, extended setting
- Mount the SwisscomLab package; call swisscom.lab.health:info
- Same image, new LAB_ENV: change without rebuilding

Prerequisites

- Exercises 2.1–2.4 done
- Logged in to ibmwebmethods.azurecr.io
- Port 5555 free
- Reference: block2/solution/capstone2-solution.sh

Deliverable
Running MSR configured by application.properties, plus a 5–10 line note: what the log files show that podman logs doesn't; first start vs. restart time; the stop exit codes.

Classification - Internal

💬 Q&A and Discussion: Capstone 2

1. *What surprised you about MSR start-up and shutdown compared with a Windows IS service?*
2. *Where in today's landscape would a container replace a VM first: Dev, Test or Prod?*
3. *Which Swisscom settings and secrets would become application.properties entries or environment variables?*

Classification - Internal

Day 1 Recap

- ✓ A container is a process: namespaces isolate, cgroups limit, layers share
- ✓ Run, inspect, limit, connect and persist containers from the command line
- ✓ The MSR 12.1 image: UID 1724, /opt/webmethods, health and metrics without a password
- ✓ "Up" is not "ready"; a clean MSR stop needs ~13 s (exit 137 when killed)
- ✓ application.properties + $env{}: one file, every environment, no rebuild

Coming up next: Day 2: build your own images (incl. a custom MSR), then Compose with MSR, UM and PostgreSQL.

Classification - Internal